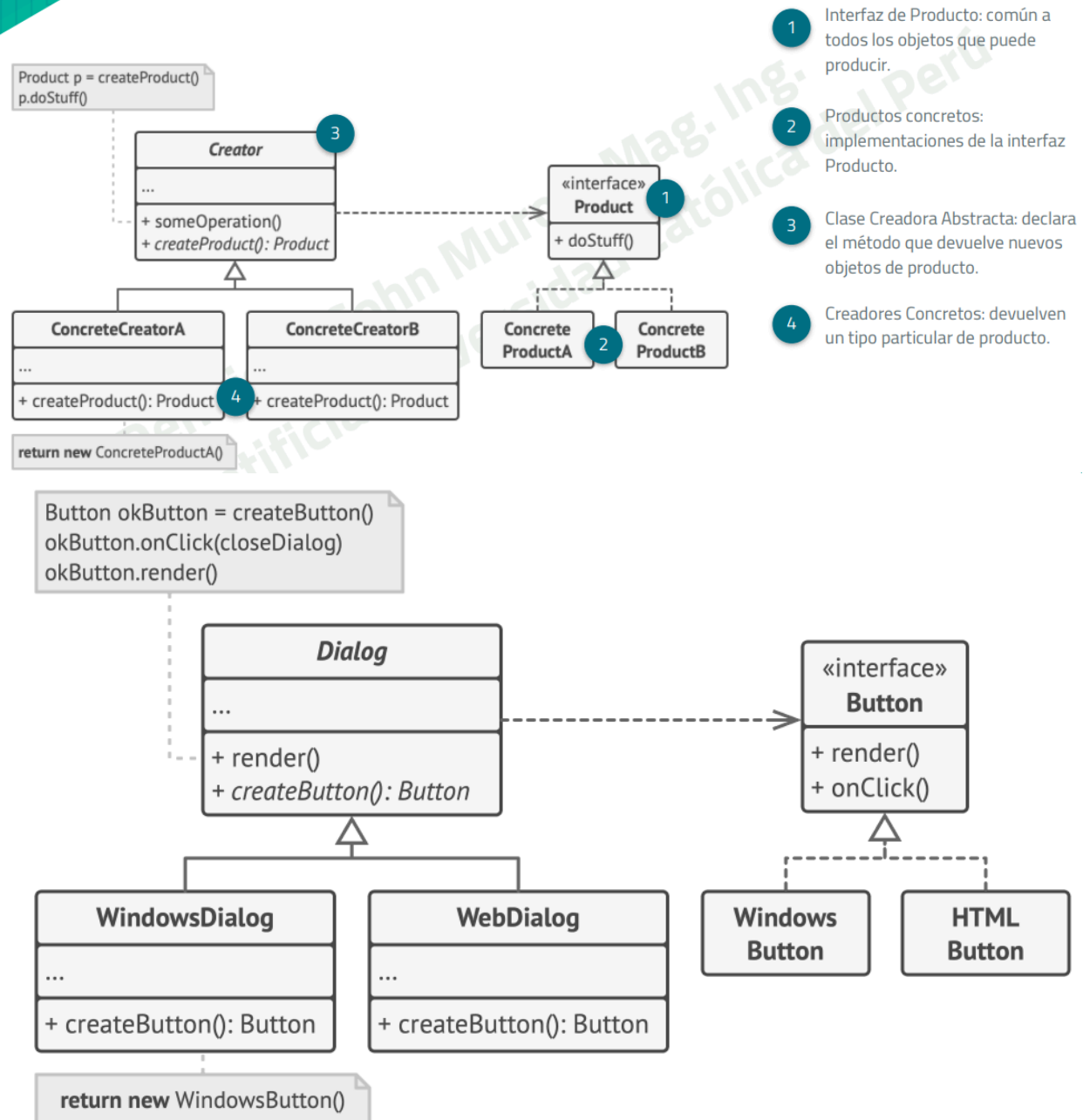
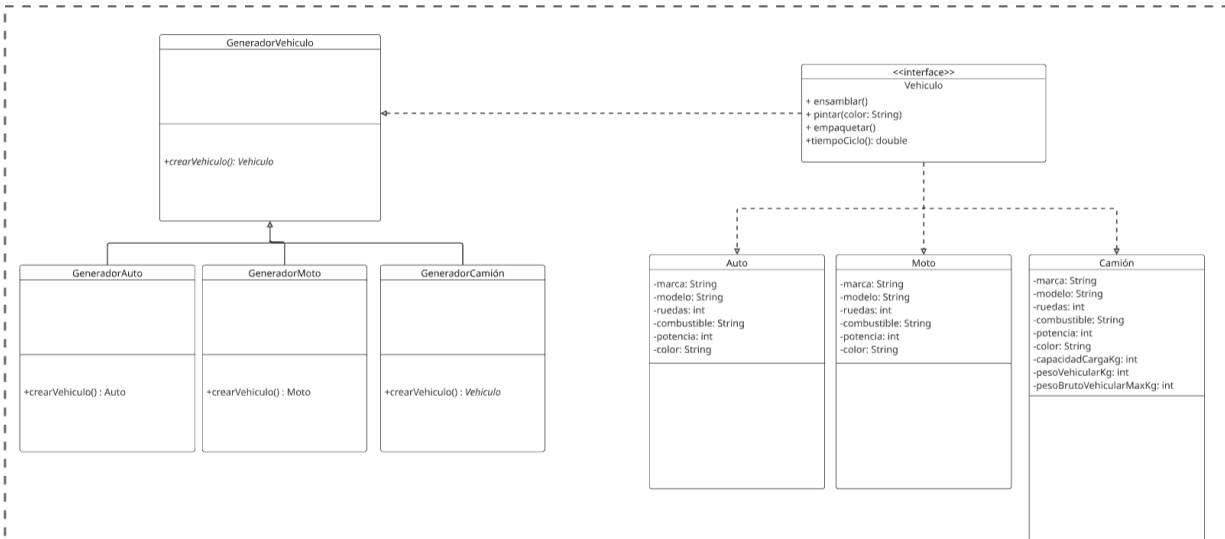


1. Factory Method (Método Fábrica)





Problema que resuelve: Permitir que las subclases decidan qué clase instanciar.

Cómo funciona: Define una interfaz para crear objetos, pero delega la creación concreta a las subclases.

Ejemplo:

```

// Producto abstracto
interface Transporte {
    void entregar();
}

// Productos concretos
class Camion implements Transporte {
    public void entregar() { System.out.println("Entregando por camión"); }
}

class Barco implements Transporte {
    public void entregar() { System.out.println("Entregando por barco"); }
}

// Creador abstracto
abstract class Logistica {
    public void planificarEntrega() {
        Transporte transporte = crearTransporte();
        transporte.entregar();
    }
}

// Factory Method
abstract Transporte crearTransporte();
}

// Creadores concretos
class LogisticaTerrestre extends Logistica {
    Transporte crearTransporte() {

```

```
        return new Camion();
    }
}

class LogisticaMaritima extends Logistica {
    Transporte crearTransporte() {
        return new Barco();
    }
}
```

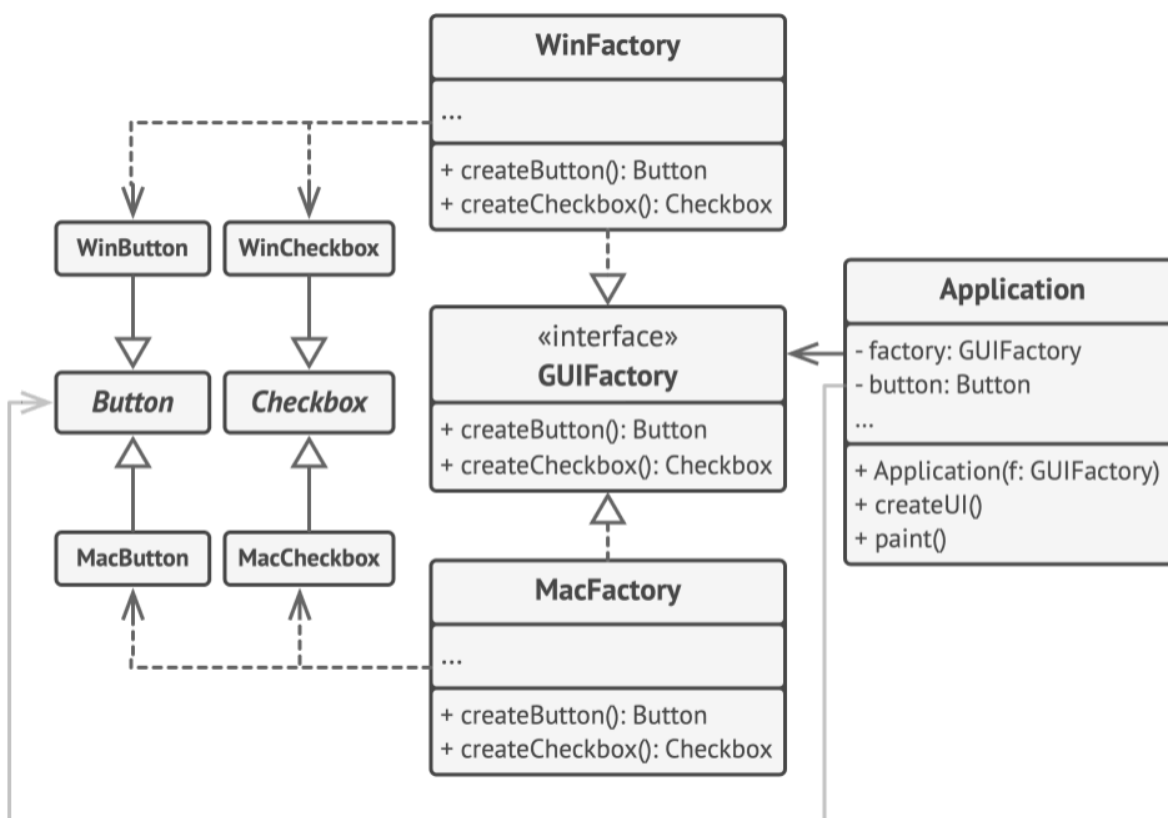
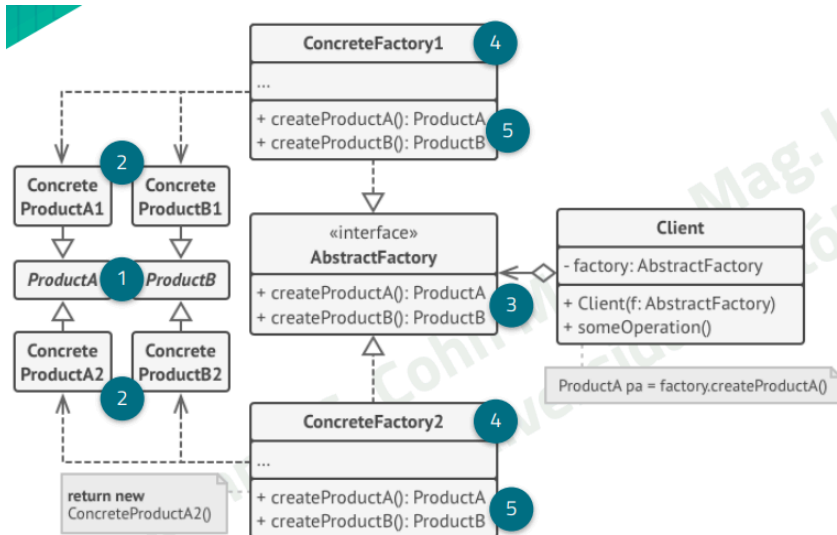
Cuándo usarlo:

-  Cuando no sabes exactamente qué tipo de producto necesitarás
-  Cuando quieres que las subclases decidan qué crear
-  Para frameworks que necesitan extensibilidad

Consecuencias:

-  Separa creación de uso
-  Fácil extensibilidad
-  Cumple Open/Closed Principle

2. Abstract Factory (Fábrica Abstracta)



```
// Familias de productos
interface Boton {
    void render();
}

interface Ventana {
    void mostrar();
}
```

```

// Productos concretos para Windows
class BotonWindows implements Boton {
    public void render() { System.out.println("Renderizando botón Windows"); }
}

class VentanaWindows implements Ventana {
    public void mostrar() { System.out.println("Mostrando ventana Windows"); }
}

// Productos concretos para Mac
class BotonMac implements Boton {
    public void render() { System.out.println("Renderizando botón Mac"); }
}

class VentanaMac implements Ventana {
    public void mostrar() { System.out.println("Mostrando ventana Mac"); }
}

// Abstract Factory
interface GUIFactory {
    Boton crearBoton();
    Ventana crearVentana();
}

// Factories concretas
class WindowsFactory implements GUIFactory {
    public Boton crearBoton() { return new BotonWindows(); }
    public Ventana crearVentana() { return new VentanaWindows(); }
}

class MacFactory implements GUIFactory {
    public Boton crearBoton() { return new BotonMac(); }
    public Ventana crearVentana() { return new VentanaMac(); }
}

// Cliente
class Aplicacion {
    private GUIFactory factory;
    private Boton boton;
    private Ventana ventana;

    public Aplicacion(GUIFactory factory) {
        this.factory = factory;
        this.boton = factory.crearBoton();
        this.ventana = factory.crearVentana();
    }
}

```

Cuándo usarlo:

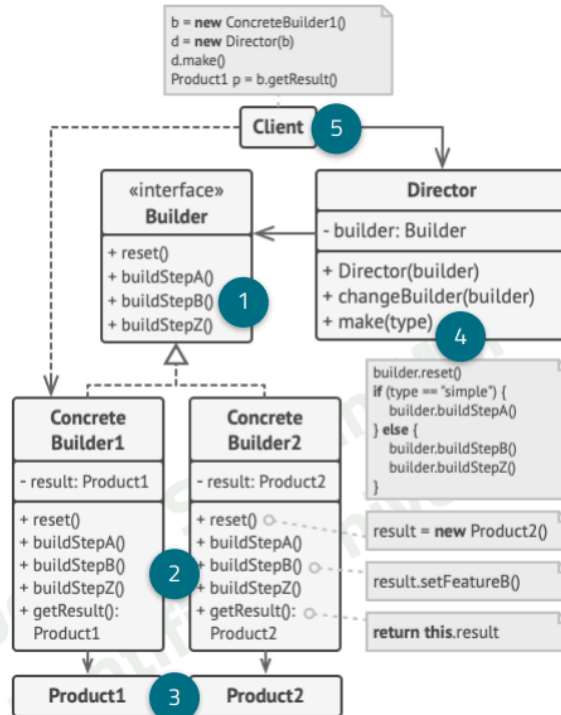
-  Sistemas que deben ser independientes de cómo se crean sus productos
-  Familias de productos relacionados
-  Intercambiar fácilmente entre familias

Consecuencias:

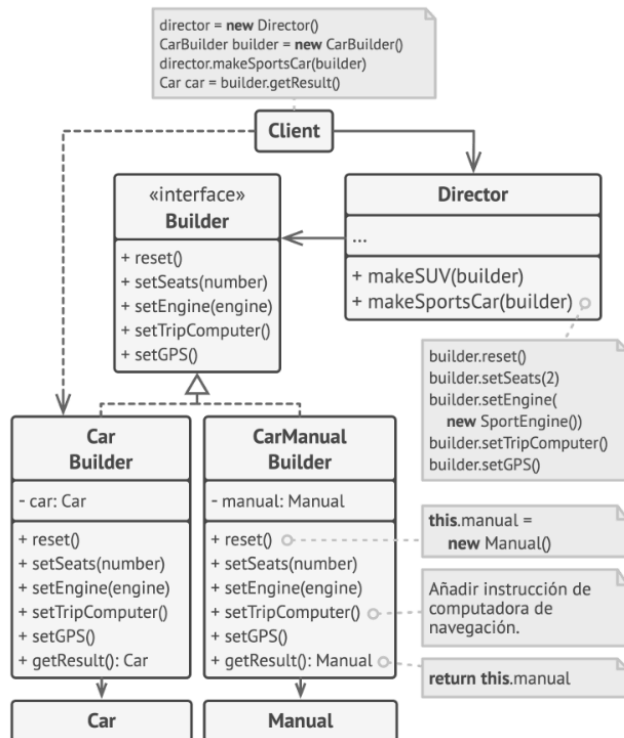
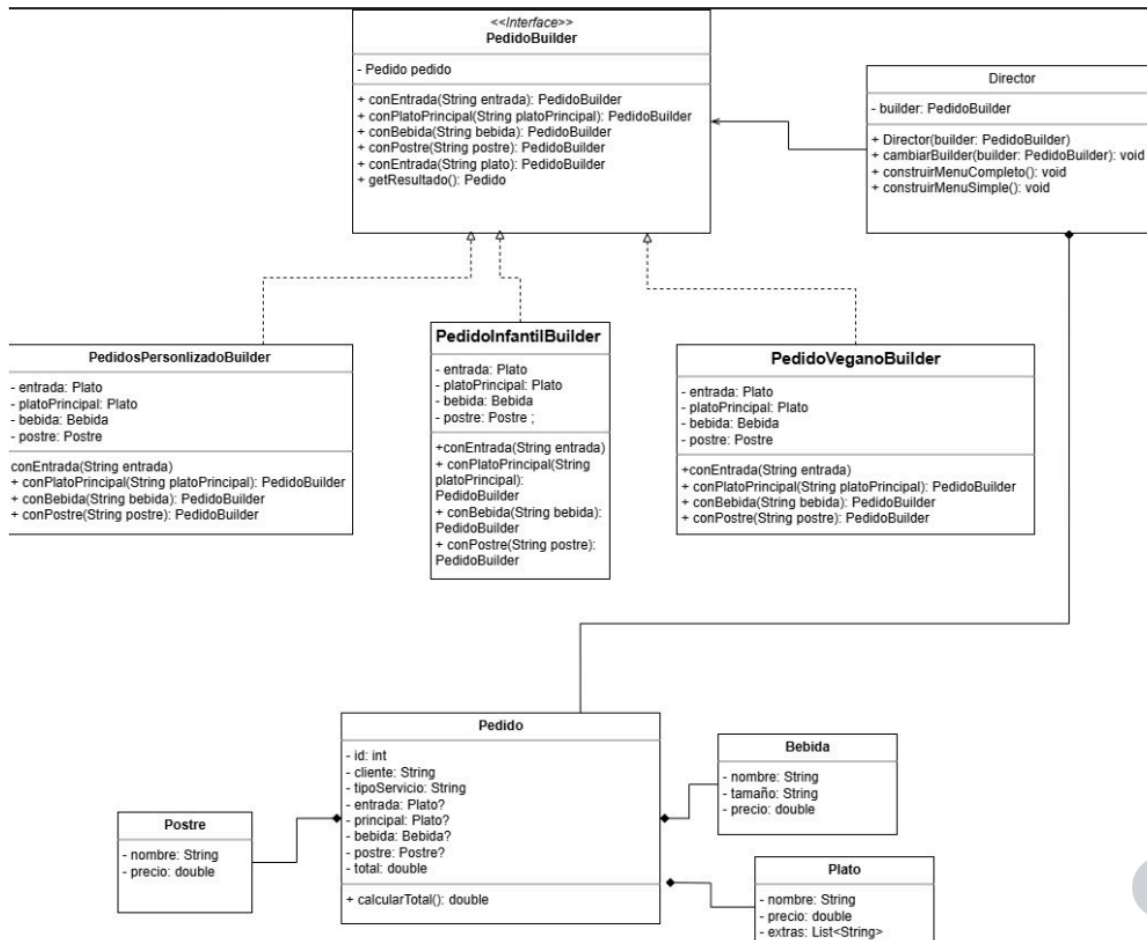
-  Consistencia entre productos
-  Fácil intercambio de familias

- — Difícil añadir nuevos tipos de productos

3. Builder (Constructor)



- 1 Interfaz Constructora declara pasos de construcción comunes.
- 2 Constructores Concretos implementan los pasos de construcción.
- 3 Los Productos son los objetos resultantes. No heredan de los Constructores
- 4 Clase Directora define el orden de los pasos de construcción
- 5 El Cliente define el Constructor a utilizar.



Problema que resuelve: Construir objetos complejos paso a paso.

Cómo funciona: Separa la construcción de un objeto complejo de su representación.

Ejemplo:

```
// Producto complejo
class Casa {
    private int habitaciones;
    private int banos;
    private boolean jardin;
    private boolean garaje;
    private boolean piscina;

    // Constructor privado
    private Casa() {}

    // Getters...

    // Builder
    public static class Builder {
        private Casa casa;

        public Builder() {
            casa = new Casa();
        }

        public Builder setHabitaciones(int num) {
            casa.habitaciones = num;
            return this;
        }

        public Builder setBanos(int num) {
            casa.banos = num;
            return this;
        }

        public Builder conJardin() {
            casa.jardin = true;
            return this;
        }

        public Builder conGaraje() {
            casa.garaje = true;
            return this;
        }

        public Builder conPiscina() {
            casa.piscina = true;
            return this;
        }

        public Casa build() {
            return casa;
        }
    }
}

// Uso
Casa mansion = new Casa.Builder()
```

```

.setHabitaciones(5)
.setBanos(3)
.conJardin()
.conGaraje()
.conPiscina()

```

```

.build();

```

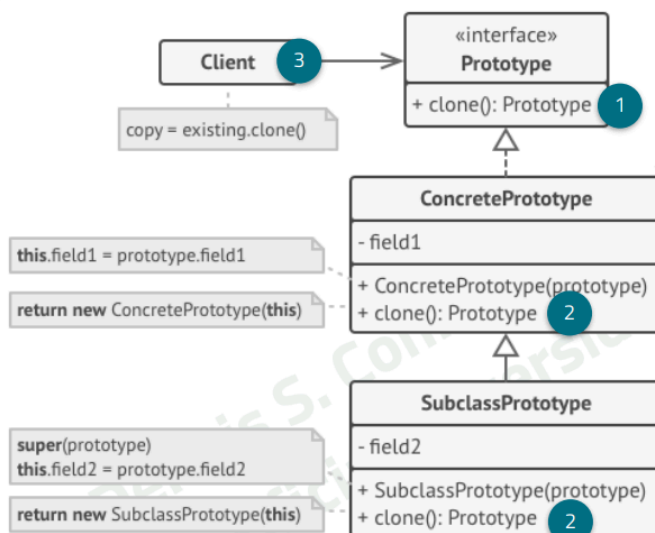
Cuándo usarlo:

- ☒ Objetos con muchos parámetros opcionales
- ☒ Construcción paso a paso
- ☒ Diferentes representaciones del mismo objeto

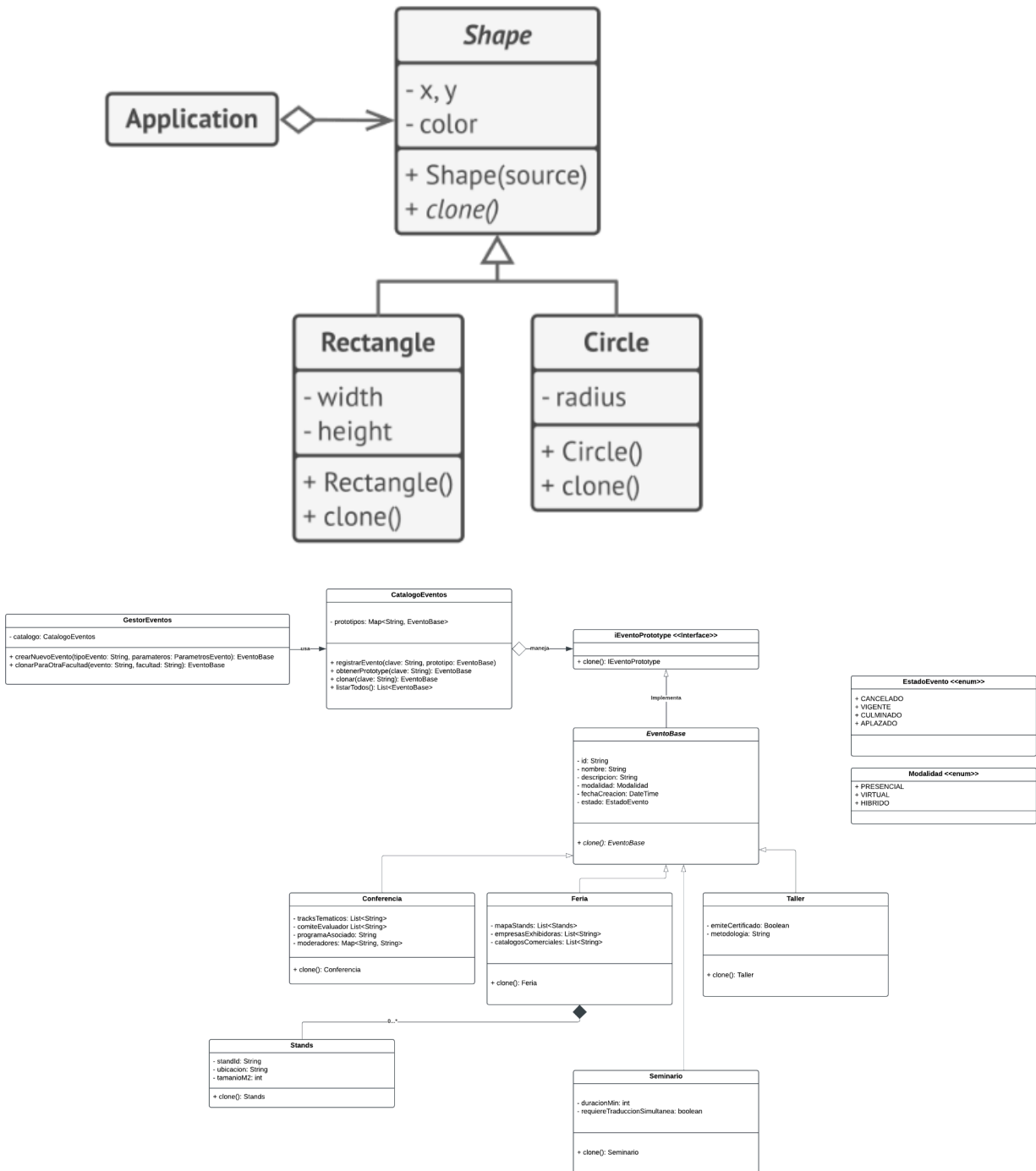
Consecuencias:

- ☒ Control granular sobre construcción
- ☒ Código más legible
- ☒ Reutilización de pasos de construcción

4. Prototype (Prototipo)



- 1 Interfaz Prototipo declara los métodos de clonación.
- 2 Prototipo Concreto implementa el método de clonación. Copia la información del objeto original al clon.
- 3 Cliente puede producir una copia de cualquier objeto que siga la interfaz del prototipo.



Problema que resuelve: Crear nuevos objetos copiando existentes sin conocer su estructura interna.

Cómo funciona: Delega la clonación a los propios objetos mediante un método `clone()`.

Ejemplo:

```
interface Prototype extends Cloneable {

    Prototype clone();

}

class Documento implements Prototype {
    private String titulo;
    private String contenido;
    private List<String> imagenes;

    public Documento(String titulo, String contenido, List<String> imagenes) {
        this.titulo = titulo;
        this.contenido = contenido;
        this.imagenes = new ArrayList<>(imagenes);
    }

    // Constructor de copia
    private Documento(Documento original) {
        this.titulo = original.titulo;
        this.contenido = original.contenido;
        this.imagenes = new ArrayList<>(original.imagenes); // Deep copy
    }

    public Prototype clone() {
        return new Documento(this);
    }

    // Métodos para modificar...
}

// Uso
Documento original = new Documento("Contrato", "Texto...", Arrays.asList("img1", "img2"));

Documento copia = (Documento) original.clone();
```

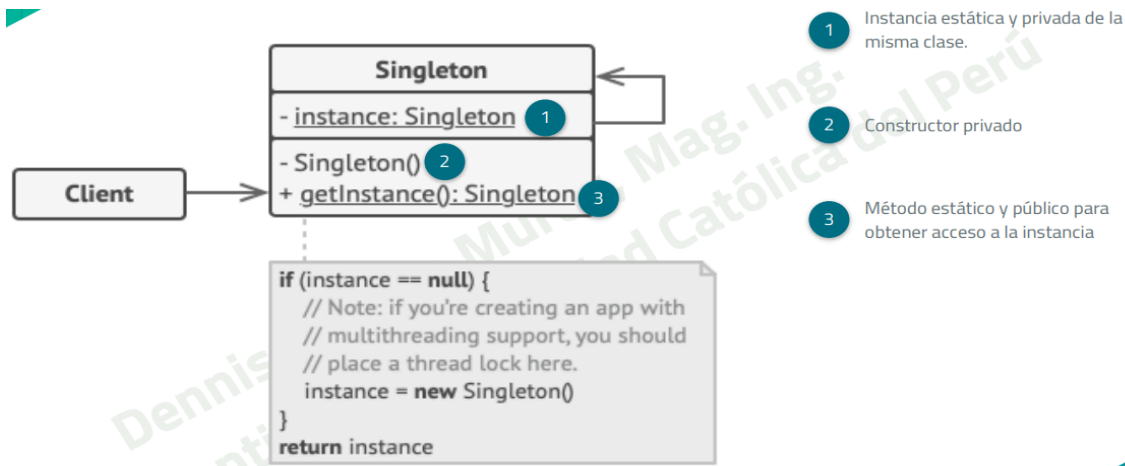
Cuándo usarlo:

-  Cuando la creación es más costosa que la copia
-  Evitar jerarquías de fábricas
-  Sistemas que crean objetos en tiempo de ejecución

Consecuencias:

-  Evita jerarquías complejas de creación
-  Clonación más eficiente que creación
-  Complejo con referencias circulares

5. Singleton (Instancia Única)



Problema que resuelve: Garantizar que una clase tenga solo una instancia y proporcionar acceso global.

Cómo funciona: Constructor privado y método estático para obtener la instancia.

Ejemplo:

```
class Configuracion {  
    private static Configuracion instancia;  
    private String urlBaseDatos;  
    private int tiempoEspera;  
  
    // Constructor privado  
    private Configuracion() {  
        // Cargar configuración desde archivo...  
        urlBaseDatos = "jdbc:mysql://localhost:3306/mydb";  
        tiempoEspera = 30;  
    }  
  
    // Método estático para obtener instancia  
    public static Configuracion getInstance() {  
        if (instancia == null) {  
            instancia = new Configuracion();  
        }  
        return instancia;  
    }  
  
    // Getters y setters...  
}
```

```
// Uso
Configuracion config = Configuracion.getInstance();

String url = config.getUrlBaseDatos();
```

Cuándo usarlo:

-  Acceso a recursos compartidos (base de datos, archivos)
-  Configuración global
-  Logger, cache

Consecuencias:

-  Acceso controlado a instancia única
-  Ahorro de recursos
-  Puede enmascarar mal diseño
-  Dificulta testing



Guía de Selección Rápida

Patrón	Usar cuando...	Ejemplo común
Factory Method	Las subclases deciden qué crear	Frameworks, plugins
Abstract Factory	Familias de objetos relacionados	GUI multiplataforma
Builder	Objetos complejos con muchos parámetros	Construcción de documentos
Prototype	Clonación es más eficiente que creación	Editores gráficos
Singleton	Una única instancia global	Configuración, Logger